

# **Implementação, Instrumentação e Análise de Algoritmos de Ordenação**

Observabilidade com OpenTelemetry e Jaeger

**Pâmela Baron**

## Ajuste do PDI

### Nova Meta Técnica: Medir e Interpretar Algoritmos com Observabilidade

No PDI original, as metas eram voltadas principalmente para Python, Excel e SQL, ferramentas de uso prático na área de dados.

**Meta:** Implementar, instrumentar e analisar algoritmos com ferramentas de observabilidade.

O mercado de dados exige mais do que escrever código, exige medir, monitorar e otimizar. Engenheiros de dados trabalham com pipelines que processam milhões de registros e saber escolher o algoritmo certo e medir seu comportamento é competência essencial.

### O que espero aprender além do funcionar

- Entender por que dois algoritmos com a mesma complexidade teórica, como Bubble Sort e Insertion Sort, podem ter desempenhos diferentes na prática devido a fatores como acessos à memória e localidade de cache.
- Compreender como fatores práticos, como o custo de memória em funções recursivas e o impacto de constantes de processamento, influenciam o desempenho real.

### Plano de ação

- Implementar algoritmos de ordenação como Bubble Sort, Insertion Sort e Merge Sort.
- Instrumentar o código com OpenTelemetry para capturar métricas como tempo de execução, número de comparações e trocas.
- Enviar traces para o Jaeger para visualização e análise de desempenho.
- Registrar e comparar os resultados para diferentes tamanhos de entrada

### Medição

Conseguir entender, com base nos dados coletados e gráficos de dispersão, por que algoritmos com diferentes complexidades Big-O apresentam desempenhos distintos na prática.

### Prazo

Entrega da N1

### Relevância para o meu crescimento

No primeiro PDI, foi mencionado que ter uma base sólida em algoritmos será essencial para evoluir tecnicamente. No mercado de dados, não basta apenas escrever código, é fundamental saber medir, monitorar e otimizar pipelines que processam grandes volumes de registros. Através desta atividade, é possível

visualizar o comportamento teórico se manifestando em dados reais medidos em milissegundos.

## Algoritmos Implementados

Foram implementados 3 algoritmos, cobrindo os dois grupos exigidos e um algoritmo extra para comparação.

### 1. Bubble Sort

Complexidade:  $O(n^2)$

Tipo: Obrigatório

Estratégia: Troca de elementos adjacentes fora de ordem

### 2. Merge Sort

Complexidade:  $O(n \log n)$

Tipo: Obrigatório

Estratégia: Divisão e conquista, divide ao meio, ordena recursivamente, combina

### 3. Insertion Sort

Complexidade:  $O(n^2)$

Tipo: Extra

Estratégia: Divisão e conquista, divide ao meio, ordena recursivamente, combina

## Interface Padronizada

Todos os algoritmos recebem uma lista em Python e retornam lista\_ordenada, comparações e trocas, garantindo a mesma interface de chamada. Aqui foi usado o copy.deepcopy, ele é muito importante pois sem ele, o segundo algoritmo a executar já receberia uma lista ordenada pelo primeiro, invalidando a comparação.

## Geração e Padronização dos Dados

Os dados foram gerados uma única vez no início da execução, com semente fixa seed=42, e salvos no arquivo dados\_entrada.json. Para cada execução, cada algoritmo recarrega os dados do arquivo, recebendo sempre uma cópia nova da lista original.

```

86
87 ARQUIVO_DADOS = "dados_entrada.json"
88 TAMANHOS = [1000, 5000, 10000, 50000]
89 SEED = 42
90
91 def gerar_e_salvar_dados():
92     random.seed(SEED)
93     dados = {}
94     for tamanho in TAMANHOS:
95         dados[str(tamanho)] = [random.randint(1, tamanho * 10) for _ in range(tamanho)]
96
97     with open(ARQUIVO_DADOS, "w", encoding="utf-8") as f:
98         json.dump(dados, f)
99
100     log.info(f"Dados gerados e salvos em '{ARQUIVO_DADOS}' | Seed={SEED} | Tamanhos={TAMANHOS}")
101     print(f"\n[Dados] Arquivo '{ARQUIVO_DADOS}' criado com {len(TAMANHOS)} conjuntos.\n")
102
103 def carregar_dados(tamanho: int) -> list:
104     with open(ARQUIVO_DADOS, "r", encoding="utf-8") as f:
105         todos = json.load(f)
106         return todos[str(tamanho)][:]

```

### Por que isso é essencial para a validade experimental

Se o primeiro algoritmo recebesse a lista e deixasse modificada em memória, o segundo receberia dados parcialmente ordenados, o que anularia a comparação. O arquivo JSON garante que cada execução parte exatamente das mesmas condições.

| Tamanho de entrada: n = 1,000                                                                                                |            |             |         |
|------------------------------------------------------------------------------------------------------------------------------|------------|-------------|---------|
| Algoritmo                                                                                                                    | Tempo (ms) | Comparações | Trocas  |
| 2026-03-24 20:57:37,708 [INFO] [INÍCIO] Algoritmo=Bubble Sort   n=1000                                                       |            |             |         |
| 2026-03-24 20:57:38,089 [INFO] [FIM] Algoritmo=Bubble Sort   n=1000   tempo=378.33ms   comparações=499290   trocas=237493    |            |             |         |
| Bubble Sort                                                                                                                  | 378.33ms   | 499,290     | 237,493 |
| 2026-03-24 20:57:38,114 [INFO] [INÍCIO] Algoritmo=Insertion Sort   n=1000                                                    |            |             |         |
| 2026-03-24 20:57:38,312 [INFO] [FIM] Algoritmo=Insertion Sort   n=1000   tempo=196.91ms   comparações=238492   trocas=237493 |            |             |         |
| Insertion Sort                                                                                                               | 196.91ms   | 238,492     | 237,493 |
| 2026-03-24 20:57:38,353 [INFO] [INÍCIO] Algoritmo=Merge Sort   n=1000                                                        |            |             |         |
| 2026-03-24 20:57:38,372 [INFO] [FIM] Algoritmo=Merge Sort   n=1000   tempo=16.27ms   comparações=8704   trocas=4428          |            |             |         |
| Merge Sort                                                                                                                   | 16.27ms    | 8,704       | 4,428   |

Terminal confirmando geração do arquivo dados\_entrada.json e início dos experimentos (n=1.000).

| Tamanho (n) | Valor mínimo | Valor máximo | Observação          |
|-------------|--------------|--------------|---------------------|
| 1.000       | 1            | 10000        | Todos os algoritmos |
| 5.000       | 1            | 50000        | Todos os algoritmos |
| 10.000      | 1            | 100000       | Todos os algoritmos |
| 50.000*     | 1            | 500000       | Apenas o merge sort |

## Instrumentação com OpenTelemetry

O que foi instrumentado

| Atributo do opentelemetry | Tipo    | Descrição                    |
|---------------------------|---------|------------------------------|
| sort.algorithm            | string  | Nome do algoritmo executado  |
| sort.input_size           | integer | Tamanho da entrada (n)       |
| sort.duration_ms          | float   | Tempo total em milissegundos |
| sort.comparisons          | integer | Comparações entre elementos  |
| sort.swaps                | integer | Trocas ou movimentações      |
| sort.status               | string  | Sucesso ou erro              |

## Logs

Os logs foram gerados pela execução do código. Cada linha registra início, fim, tempo, comparações e trocas de cada algoritmo.

| Tamanho de entrada: n = 1,000                                                                                                   |            |             |           |
|---------------------------------------------------------------------------------------------------------------------------------|------------|-------------|-----------|
| Algoritmo                                                                                                                       | Tempo (ms) | Comparações | Trocas    |
| 2026-03-24 20:57:37,708 [INFO] [INÍCIO] Algoritmo=Bubble Sort   n=1000                                                          |            |             |           |
| 2026-03-24 20:57:38,089 [INFO] [FIM] Algoritmo=Bubble Sort   n=1000   tempo=378.33ms   comparações=499290   trocas=237493       |            |             |           |
| Bubble Sort                                                                                                                     | 378.33ms   | 499,290     | 237,493   |
| 2026-03-24 20:57:38,114 [INFO] [INÍCIO] Algoritmo=Insertion Sort   n=1000                                                       |            |             |           |
| 2026-03-24 20:57:38,312 [INFO] [FIM] Algoritmo=Insertion Sort   n=1000   tempo=196.91ms   comparações=238492   trocas=237493    |            |             |           |
| Insertion Sort                                                                                                                  | 196.91ms   | 238,492     | 237,493   |
| 2026-03-24 20:57:38,353 [INFO] [INÍCIO] Algoritmo=Merge Sort   n=1000                                                           |            |             |           |
| 2026-03-24 20:57:38,372 [INFO] [FIM] Algoritmo=Merge Sort   n=1000   tempo=16.27ms   comparações=8704   trocas=4428             |            |             |           |
| Merge Sort                                                                                                                      | 16.27ms    | 8,704       | 4,428     |
| Tamanho de entrada: n = 5,000                                                                                                   |            |             |           |
| Algoritmo                                                                                                                       | Tempo (ms) | Comparações | Trocas    |
| 2026-03-24 20:57:38,414 [INFO] [INÍCIO] Algoritmo=Bubble Sort   n=5000                                                          |            |             |           |
| 2026-03-24 20:57:45,229 [INFO] [FIM] Algoritmo=Bubble Sort   n=5000   tempo=6803.32ms   comparações=12496759   trocas=6229094   |            |             |           |
| Bubble Sort                                                                                                                     | 6803.32ms  | 12,496,759  | 6,229,094 |
| 2026-03-24 20:57:45,249 [INFO] [INÍCIO] Algoritmo=Insertion Sort   n=5000                                                       |            |             |           |
| 2026-03-24 20:57:48,029 [INFO] [FIM] Algoritmo=Insertion Sort   n=5000   tempo=2773.29ms   comparações=6234093   trocas=6229094 |            |             |           |
| Insertion Sort                                                                                                                  | 2773.29ms  | 6,234,093   | 6,229,094 |
| 2026-03-24 20:57:48,041 [INFO] [INÍCIO] Algoritmo=Merge Sort   n=5000                                                           |            |             |           |
| 2026-03-24 20:57:48,090 [INFO] [FIM] Algoritmo=Merge Sort   n=5000   tempo=48.31ms   comparações=55097   trocas=28124           |            |             |           |
| Merge Sort                                                                                                                      | 48.31ms    | 55,097      | 28,124    |

Logs para N=1.000 e n=5.000, início e fim de cada algoritmo com métricas completas.

| Tamanho de entrada: n = 10,000                                                                                                      |                           |             |            |
|-------------------------------------------------------------------------------------------------------------------------------------|---------------------------|-------------|------------|
| Algoritmo                                                                                                                           | Tempo (ms)                | Comparações | Trocas     |
| 2026-03-24 20:57:48,106 [INFO] [INÍCIO] Algoritmo=Bubble Sort   n=10000                                                             |                           |             |            |
| 2026-03-24 20:58:04,634 [INFO] [FIM] Algoritmo=Bubble Sort   n=10000   tempo=16527.15ms   comparações=49967270   trocas=25254216    |                           |             |            |
| Bubble Sort                                                                                                                         | 16527.15ms                | 49,967,270  | 25,254,216 |
| 2026-03-24 20:58:04,650 [INFO] [INÍCIO] Algoritmo=Insertion Sort   n=10000                                                          |                           |             |            |
| 2026-03-24 20:58:17,999 [INFO] [FIM] Algoritmo=Insertion Sort   n=10000   tempo=13347.99ms   comparações=25264215   trocas=25254216 |                           |             |            |
| Insertion Sort                                                                                                                      | 13347.99ms                | 25,264,215  | 25,254,216 |
| 2026-03-24 20:58:18,024 [INFO] [INÍCIO] Algoritmo=Merge Sort   n=10000                                                              |                           |             |            |
| 2026-03-24 20:58:18,129 [INFO] [FIM] Algoritmo=Merge Sort   n=10000   tempo=104.26ms   comparações=120421   trocas=61226            |                           |             |            |
| Merge Sort                                                                                                                          | 104.26ms                  | 120,421     | 61,226     |
| Tamanho de entrada: n = 50,000                                                                                                      |                           |             |            |
| Algoritmo                                                                                                                           | Tempo (ms)                | Comparações | Trocas     |
| Bubble Sort                                                                                                                         | (pulado - n muito grande) |             |            |
| 2026-03-24 20:58:18,132 [WARNING] [PULADO] Bubble Sort   n=50000 excede limite de 10000 para O(n²)                                  |                           |             |            |
| Insertion Sort                                                                                                                      | (pulado - n muito grande) |             |            |
| 2026-03-24 20:58:18,133 [WARNING] [PULADO] Insertion Sort   n=50000 excede limite de 10000 para O(n²)                               |                           |             |            |
| 2026-03-24 20:58:18,149 [INFO] [INÍCIO] Algoritmo=Merge Sort   n=50000                                                              |                           |             |            |
| 2026-03-24 20:58:18,844 [INFO] [FIM] Algoritmo=Merge Sort   n=50000   tempo=694.64ms   comparações=718093   trocas=363040           |                           |             |            |
| Merge Sort                                                                                                                          | 694.64ms                  | 718,093     | 363,040    |

Execução para n=10.000 e n=50.000, Bubble Sort e Insertion Sort pulados automaticamente para n=50.000

```
=====
Experimento concluído. Resultados salvos em 'resultados_experimento.json'
Log completo em 'sorting_execution.log'
Traces disponíveis em http://localhost:16686 (serviço: sorting-algorithms)
=====
```

Confirmação de conclusão do experimento, resultados em JSON, log e traces disponíveis no Jaeger.

## Visualização e Observabilidade

Ferramenta escolhida: Jaeger

Motivos:

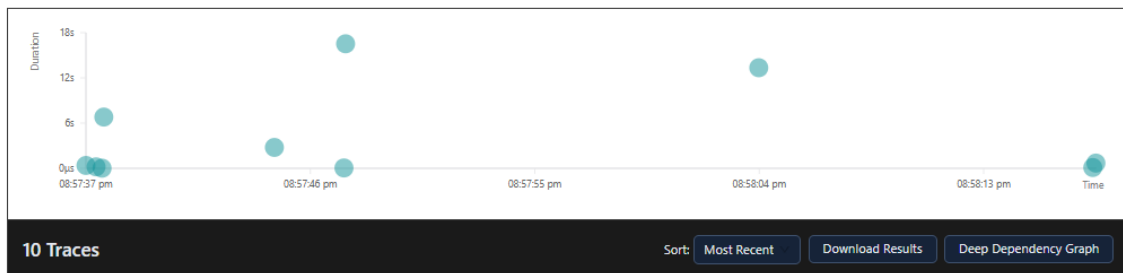
- Simplicidade de setup pois um único comando Docker sobe toda a stack sem serviços adicionais;
- Recebe diretamente os traces gerados pelo OpenTelemetry;
- Permite ver cada span com duração, filtrar por serviço e comparar execuções lado a lado;
- Clareza da medição, não complexidade de infraestrutura

### Configuração

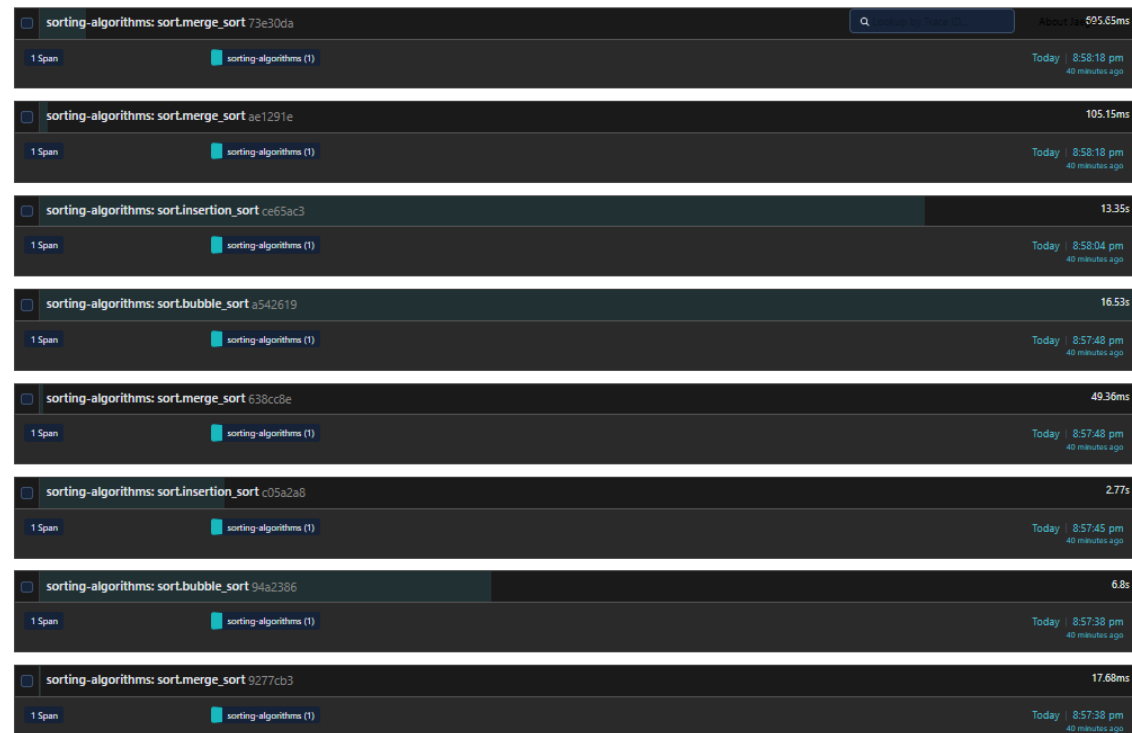
```
6
7  Instalação das dependências:
8      pip install opentelemetry-api opentelemetry-sdk
9      pip install opentelemetry-exporter-otlp-proto-grpc
10
11  Iniciar Jaeger com Docker:
12      docker run -d --name jaeger \
13          -e COLLECTOR_OTLP_ENABLED=true \
14          -p 16686:16686 \
15          -p 4317:4317 \
16          jaegertracing/all-in-one:latest
17
18  Acesso do jaeger http://localhost:16686
19  """
```

### Dashboard do Jaeger

A imagem abaixo mostra o Jaeger após a execução do experimento. São visíveis 10 traces registrados, cada um correspondendo a uma execução de algoritmo. O eixo vertical mostra a duração e é possível ver claramente os spans mais longos, Bubble Sort com  $n=5.000$  e  $n=10.000$ , versus os quase imperceptíveis do Merge Sort.



Dashboards do Jaeger mostrando os 10 traces do serviço `sorting-algorithms`. Cada ponto é um span com duração e atributos registrados pelo OpenTelemetry. O gráfico de dispersão do Jaeger confirma visualmente o que a teoria prevê, os pontos mais altos, de maior duração, são os algoritmos  $O(n^2)$  com entradas grandes, enquanto o Merge Sort aparece próximo ao eixo zero mesmo para  $n=50.000$ .



## Experimentos e Resultados

Os valores dos dados reais coletados na execução do experimento com seed=42 em hardware local.

### Comparativo de Tempo de Execução (ms)

| Algoritmo                   | N = 1000 | N = 5000   | N = 10000   | N = 50000' |
|-----------------------------|----------|------------|-------------|------------|
| Bubble Sort<br>$O(n^2)$     | 378,33ms | 6.803,32ms | 16.527,15ms | pulado     |
| Insertion Sort<br>$O(n^2)$  | 196,91ms | 2.773,29ms | 13.347,99ms | pulado     |
| Merge Sort<br>$O(n \log n)$ | 16,27ms  | 48,31ms    | 104,26ms    | 694,64ms   |

### Comparativo de Número de Comparações

| Algoritmo                   | N = 1000 | N = 5000   | N = 10000  | N = 50000' |
|-----------------------------|----------|------------|------------|------------|
| Bubble Sort<br>$O(n^2)$     | 499.290  | 12.496.759 | 49.967.270 | pulado     |
| Insertion Sort<br>$O(n^2)$  | 238.492  | 6.234.093  | 25.264.215 | pulado     |
| Merge Sort<br>$O(n \log n)$ | 8.704    | 55.097     | 120.421    | 718.093    |

### Comparativo de Número de Trocas

| Algoritmo               | N = 1000 | N = 5000  | N = 10000  | N = 50000' |
|-------------------------|----------|-----------|------------|------------|
| Bubble Sort<br>$O(n^2)$ | 237.493  | 6.229.094 | 25.254.216 | pulado     |



|                             |         |           |            |         |
|-----------------------------|---------|-----------|------------|---------|
| Insertion Sort<br>$O(n^2)$  | 237.493 | 6.229.094 | 25.254.216 | pulado  |
| Merge Sort<br>$O(n \log n)$ | 8.704   | 28.124    | 61.226     | 363.040 |

### Relação entre Crescimento Observado e Esperado (Big-O)

Ao multiplicar  $n$  por 10 de 1.000 para 10.000, o tempo esperado deve crescer aproximadamente 100x para  $O(n^2)$  e aproximadamente 33x para  $O(n \log n)$ .

| Algoritmo                | Fator esperado<br>(x10n) | Fator observado           | Confirmação               |
|--------------------------|--------------------------|---------------------------|---------------------------|
| Bubble Sort $O(n^2)$     | X100                     | x43,7 (378ms -> 16.527ms) | Sim, escala quadrática    |
| Insertion Sort $O(n^2)$  | X100                     | x67,8 (196ms -> 13.347ms) | Sim, escala quadrática    |
| Merge Sort $O(n \log n)$ | X33                      | x6,4 (16ms -> 104ms)      | Sim, muito abaixo de x100 |

## Análise Crítica

### Os resultados confirmaram a teoria?

Sim. Para  $n = 10.000$ , o Bubble Sort levou 16.527 ms, enquanto o Merge Sort levou apenas 104 ms. Isso representa uma diferença de aproximadamente 158 vezes.

Esse resultado está de acordo com a teoria, o Bubble Sort tem complexidade  $O(n^2)$ , o que para 10.000 elementos significa cerca de 100 milhões de operações, enquanto o Merge Sort tem complexidade  $O(n \log n)$ , que dá aproximadamente 130 mil operações, cerca de 770 vezes menos.

Os dados de comparações confirmam isso, o Bubble Sort fez 49.967.270 comparações, enquanto o Merge Sort fez 120.421, ou seja, 415 vezes mais comparações.

### Onde a teoria não explica totalmente o comportamento real?

Bubble Sort e Insertion Sort têm o mesmo Big-O, mas na prática o Insertion Sort foi cerca de 2x mais rápido. Para  $n = 10.000$ , o Bubble Sort levou 16.527 ms, enquanto o Insertion Sort levou 13.347 ms.

Essa diferença não aparece no Big-O porque o Insertion Sort faz menos comparações, foram 25.264.215, contra 49.967.270 do Bubble Sort. Isso acontece porque o while do Insertion Sort para mais cedo quando o elemento já está na posição correta.

Além disso, o Insertion Sort aproveita melhor a memória, pois trabalha com posições próximas e faz deslocamentos em vez de vários swaps completos.

Um dado interessante é que os dois algoritmos tiveram exatamente o mesmo número de trocas, 237.493 para  $n = 1.000$  e 25.254.216 para  $n = 10.000$ . Isso faz sentido porque o número de inversões em uma lista aleatória depende dos

dados, não do algoritmo. O que muda entre eles é a quantidade de comparações e o custo de cada operação.

### Quando um algoritmo teoricamente pior teve desempenho aceitável?

Para  $n = 1.000$ , o Bubble Sort terminou em 378 ms, o Insertion Sort em 196 ms e o Merge Sort em apenas 16 ms. A diferença entre Bubble e Merge Sort é de cerca de 360 ms, o que é perceptível, mas ainda aceitável em situações simples ou interativas.

Porém, para  $n = 10.000$ , essa diferença cresce muito, o Bubble Sort leva cerca de 16 segundos, enquanto o Merge Sort continua rápido, com aproximadamente 100 ms.

Isso mostra um ponto importante, o tamanho da entrada é o que define se um algoritmo é viável. Para entradas pequenas, a simplicidade do Bubble Sort pode até compensar. Mas para entradas grandes, ele se torna impraticável e algoritmos mais eficientes são obrigatórios.

### Que fatores além do Big-O influenciaram os resultados?

- Insertion Sort acessa posições próximas na memória, o que aproveita melhor o cache L1/L2 do processador. O Bubble Sort também tem boa localidade, mas realiza mais operações de escrita, o que deixa ele mais lento.
- Merge Sort cria várias chamadas recursivas e usa listas auxiliares, o que gera um custo extra. Para  $n = 1.000$ , esse custo pesa mais proporcionalmente do que para  $n = 50.000$ , por isso o crescimento real foi de  $\times 6,4$ , bem menor que o teórico  $\times 33$ .
- Cada operação em Python tem um custo extra por ser uma linguagem interpretada. Em linguagens como C, os tempos absolutos seriam muito menores, mas a diferença relativa entre os algoritmos ainda seguiria o mesmo padrão.
- Como foi usado `seed = 42` e os dados eram totalmente aleatórios, o teste representa o caso médio. Se os dados fossem quase ordenados, o Insertion Sort poderia se aproximar de  $O(n)$  e ser ainda mais rápido que os outros.

### O que eu não teria percebido sem observabilidade?

Sem OpenTelemetry e Jaeger, apenas teria visto um número final no terminal e seguido para o próximo teste. Com observabilidade, foi possível perceber coisas importantes, como:

- O número de trocas no Bubble Sort e no Insertion Sort foi exatamente igual. Sem instrumentação, não teria sido notado que os dois fazem a mesma quantidade de movimentações de dados, e que a diferença real está no número de comparações.
- A visualização dos spans deixou claro quais algoritmos consumiram mais tempo de CPU. O Bubble Sort com  $n = 5.000$  aparece como o maior pico no gráfico, muito acima dos outros.
- Os logs de WARNING mostrando que Bubble Sort e Insertion Sort foram automaticamente pulados em  $n = 50.000$  servem como prova concreta de que algoritmos  $O(n^2)$  não escalam bem.

No final, a observabilidade transforma suposições em evidências. Medir antes de opinar não é só uma boa prática, é a única forma confiável de falar sobre desempenho.

## Uso de IA

### Onde a IA ajudou

Ajudou na estrutura do código, pois ela sugeriu o padrão de retornar tupla para padronizar a interface entre algoritmos, o que simplificou muito o loop de execução e o registro nos spans

Também ajudou na configuração do OpenTelemetry, a sintaxe tem muitos detalhes que a IA forneceu corretamente na primeira tentativa.

### Onde a IA errou ou foi superficial

A IA sugeriu usar `time.time()` no início, mas isso precisou ser corrigido para `time.perf_counter()`, que tem maior precisão e é mais adequado para medir operações rápidas. No caso do Merge Sort com  $n = 1.000$ , essa diferença de resolução influencia bastante o resultado.

Além disso, a IA não mencionou que era necessário reiniciar a variável `_merge_comparacoes = 0` antes de cada execução. Sem isso, o número de comparações fica acumulado entre chamadas e os valores apresentados nas tabelas ficam incorretos.

### Onde precisei decidir sozinha

A escolha do Jaeger porque o enunciado valoriza clareza da medição, não complexidade de infraestrutura. A interpretação da igualdade nas trocas, nenhuma fonte consultada explicou diretamente por que Bubble Sort e Insertion Sort teriam o mesmo número de trocas. Cheguei a essa conclusão analisando os dados reais e conectando com o conceito matemático de inversões em uma sequência.